Universidad Argentina de la Empresa (UADE)
Proceso de Desarrollo de Software

UADE Beats

Iteración 4

Grupo 10

Módulo de acceso a eventos en vivo

Proceso de Desarrollo de Software · UADE

Qué cambia en esta iteración

Pasamos de un programa local con archivos TXT a un sistema cliente-servidor con base de datos

Cambio	Qué hicimos
Strategy	La regla de qué eventos ve cada rol sale del gestor a estrategias intercambiables.
Base de datos	Los TXT se reemplazan por SQLite. Cada tipo de dato tiene su repositorio y todo se guarda en una transacción.
Servidor	El gestor se expone por una API HTTP con JSON. Cada pedido lleva un token y el servidor filtra lo que ve cada rol.
Modo cliente	Otra PC corre solo la interfaz, sin datos propios, y se conecta al servidor. Ventana y consola son las mismas.
Guardado	Automático después de cada cambio. Se quitan los botones de guardar y cargar.
Contraseñas	Se guardan como hash PBKDF2 con sal por cuenta, nunca en texto plano.
Calendarios	Se puede suscribir Google Calendar o Apple Calendar a un evento. El servidor de calendarios arranca solo con el programa.
Transmisiones	Un servicio aparte en Node.js muestra un recital del artista al ingresar a un evento en curso.
Avisos	Se elimina ntfy. Queda la interfaz `Notificador` con una implementación inactiva.

Strategy: dónde está

Una interfaz con varias implementaciones; quien la usa no sabe cuál le tocó

APLICADO EN ESTA ITERACIÓN

Qué eventos ve cada rol

- Interfaz: `PoliticaVisibilidadEvento`
- Estrategias: `VisibilidadGeneral` (usuario y admin ven todo) y `VisibilidadArtista` (solo sus eventos)
- `PoliticasVisibilidad` elige la estrategia según el rol

Antes: un `if` por rol dentro de `GestorEventosEnVivo`.
Ahora: el gestor delega en una línea.

YA ESTABA

Vincular un calendario

- Interfaz: `VinculoCalendario`
- Estrategias: `VinculoGoogle` y `VinculoApple`
- La ventana elige según el botón y el gestor la ejecuta sin saber cuál es

No la cambiamos: la reconocimos como Strategy, la documentamos y le agregamos pruebas.

Visibilidad de eventos: antes y después

Usuario y administrador ven todo el catálogo; el artista solo los eventos donde participa

```
// Antes: el gestor decidía con un if por rol
private boolean esVisiblePara(Cuenta actual, Evento evento) {
    if (actual == null) return false;
    if (actual.getTipoUsuario() != TipoUsuario.ARTISTA)
        return true;
    for (Artista artista : evento.getArtistas())
        if (artista.getId() == actual.getId()) return true;
    return false;
}

// Ahora: el gestor delega
private boolean esVisiblePara(Cuenta actual, Evento evento) {
    return visibilidad.esVisible(actual, evento);
}
```

- La regla se usa en listados, búsqueda por ID, la API y los calendarios: ahora vive en **un solo lugar**.
- Se prueba sin levantar servidor ni ventana.
- Ver no es poder modificar: eso lo siguen decidiendo `Permisos` y `ControlSesion`.



En gris, las clases nuevas (paquete `gestor`)

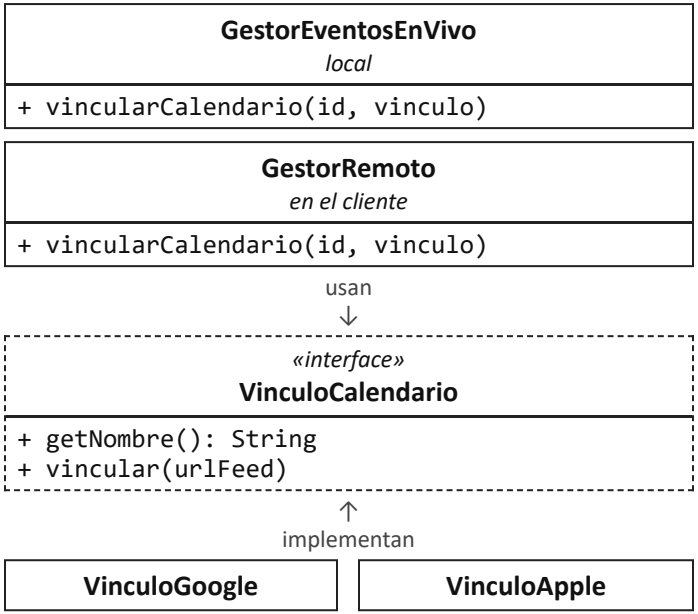
Calendarios: Google y Apple

La tarea es siempre "suscribir el calendario a este evento"; cambia cómo lo hace cada proveedor

```
// La ventana elige la estrategia según el botón
() -> vincularCalendario(new VinculoGoogle())
() -> vincularCalendario(new VinculoApple())

// El gestor la ejecuta sin saber cuál es
public void vincularCalendario(int idEvento,
                               VinculoCalendario vinculo) {
    vinculo.vincular(urlCalendario(idEvento));
}
```

- **Google:** arma el enlace de Google Calendar y abre el navegador.
- **Apple:** pasa la dirección a `webcal://` y abre la app Calendario.
- Sumar otro proveedor es una clase nueva, sin tocar el gestor.



Otros patrones aplicados

Patrón	Dónde	Para qué
Repository	`RepositorioUsuarios`, `Artistas`, `Eventos` y `Registros`	Cada uno traduce entre una tabla y los objetos. El resto no conoce SQL.
Proxy remoto	`GestorRemoto`	Misma interfaz que el gestor local; reenvía cada pedido al servidor.
Adapter	`ClienteTransmision`	Convierte las llamadas del gestor en pedidos HTTP al servicio de Node.js.
DTO	Paquete `api`	Lo que viaja por la red, ya filtrado por rol y sin contraseñas.
Inversión de dependencias	`Persistencia`, `Notificador`, `PublicadorCalendario`, `ProveedorTransmision`	El gestor recibe interfaces. SQLite reemplazó a los TXT sin tocarlo.
Segregación de interfaces	`ServicioSesion`, `ServicioConsultas`, `ServicioComandos`	Tres interfaces chicas que `ServicioEventos` reúne para las vistas.

> Revisamos también **Composite**, **Decorator** y **State** y no los aplicamos: hoy no resuelven ningún problema real del sistema.

Tres modos de ejecución

El mismo programa puede ser todo junto, solo servidor o solo cliente

Completo

```
Main
```

Datos en SQLite, API para clientes (solo desde el mismo equipo), calendarios en la red local y ventana o consola.

Servidor

```
Main servidor
```

Solo datos y API, sin interfaz. Acepta conexiones de la red. Se deja corriendo en una PC.

Cliente

```
Main cliente http://IP:8765
```

Solo interfaz. No guarda nada: inicia sesión en el servidor y trabaja con lo que su rol puede ver.

Propiedad (`-D`)	Por defecto	Uso
`uadebeats.db`	`~/.uadebeats/uadebeats.db`	Archivo de la base
`uadebeats.puerto`	`8765`	Puerto del servidor; en modo completo, uno libre si está ocupado
`uadebeats.escuchar`	`local` / `todas` en servidor	`todas` acepta conexiones de la red
`uadebeats.puerto.calendario`	`8767`	Calendarios en la red local (modo completo)
`uadebeats.url`	(ninguna)	Dirección pública https; si falta, túnel o IP de la red
`uadebeats.tunel`	(sí)	no evita abrir el túnel de Cloudflare

Arquitectura y recorrido de un pedido

Las vistas solo conocen `ServicioEventos`; el gestor solo conoce interfaces

Vistas: ventana Swing y consola

↓

`ServicioEventos` = sesión + consultas + comandos

GestorEventosEnVivo
local, con los datos

GestorRemoto
reenvía al servidor por HTTP

↓

el gestor delega en

ControlSesion + **Permisos**: quién puede hacer qué
PoliticasVisibilidad: qué eventos ve cada uno
ControlAcceso: reglas de ingreso a un evento
ControlTransmision: reglas de la transmisión
Persistencia → SQLite · **ProveedorTransmision** → Node.js

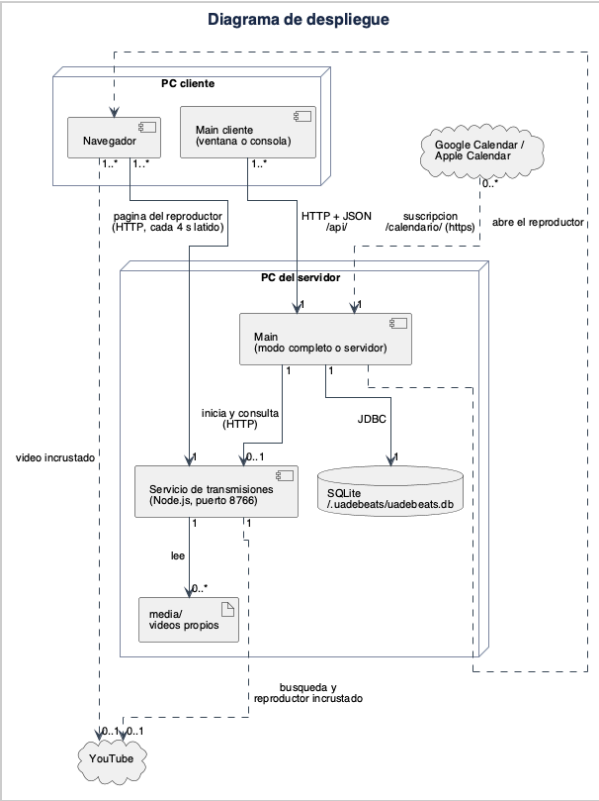
`Arranque` es el único que crea las clases concretas y se las pasa al gestor por constructor.

Un usuario pide ingresar desde otra PC

1. La ventana llama `solicitarIngreso` sobre un `GestorRemoto`.
2. Se envía `POST /api/eventos/{id}/ingresos` con el token.
3. El servidor llama al gestor real, que verifica **permiso**, **visibilidad** y **reglas de ingreso** (estado, plan, capacidad).
4. Se guarda en SQLite dentro de una transacción.
5. Si se rechaza, el cliente lanza la misma excepción que el modo local.
6. Si el evento está en curso, se abre sola la transmisión.

Despliegue y paquetes

Qué corre en cada PC y cómo se ordenó el código



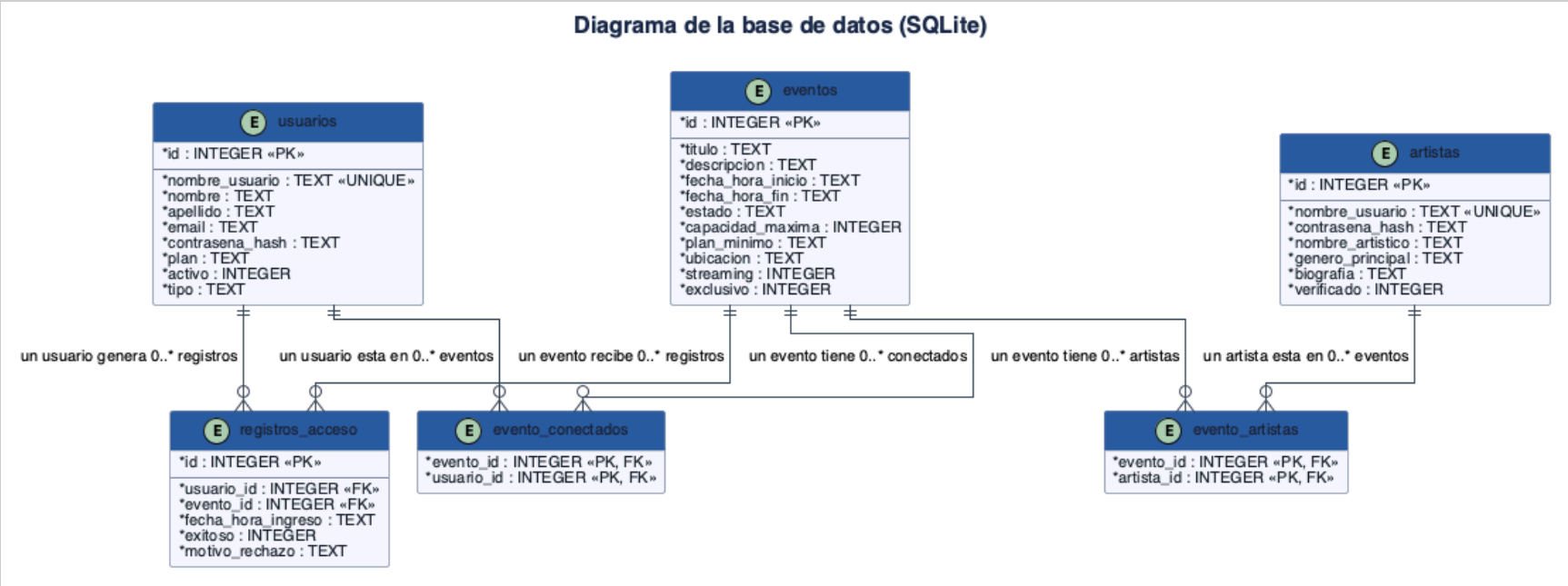
Paquete	Contenido
`modelo`	Entidades y reglas de negocio
`gestor`	Gestor, `ControlAcceso`, `ControlTransmision` y las políticas de visibilidad
`servicio`	`ServicioEventos` y sus tres partes
`seguridad`	`ControlSesion`, `Permisos`, `Accion`
`puertos`	Interfaces que recibe el gestor
`persistencia`, `calendario`, `transmision`, `notificacion`	Implementaciones de esas interfaces
`servidor`, `api`, `cliente`	API HTTP, DTO y cliente remoto
`app`, `consola`, `ui`	Arranque, menús de consola y ventanas Swing

Qué elegimos y por qué

Tema	Elegimos	En vez de	Motivo
Driver SQLite	Jar en `Codigo/lib`	Descargarlo al compilar	Compila con `javac`, sin Maven ni internet.
Acceso del cliente	Por la API del servidor	Que abra la base	SQLite no acepta conexiones de red y los permisos quedan en un solo lugar.
Formato	HTTP + JSON (gson)	Sockets con objetos serializados	Estándar, legible y no ata cliente y servidor a la misma versión.
Servidor HTTP	El del JDK	Un framework web	No agrega dependencias.
Quién guarda	El gestor, tras cada operación	Las vistas	Servidor, ventana y consola se comportan igual.
Qué ve el cliente	Copia de solo lectura que se vuelve a pedir	Repetir las reglas	Las reglas se aplican solo en el servidor.
Contraseñas	PBKDF2 del JDK, 120.000 iteraciones	bcrypt / argon2	Sin dependencias; se pueden subir las iteraciones sin invalidar hashes.
Sesiones	Token aleatorio en memoria	Usuario y contraseña en cada pedido	La contraseña viaja una sola vez.
Concurrencia	Operaciones `synchronized`	Un gestor por sesión	Un único conjunto de datos compartido.

SQLite: seis tablas

Claves foráneas activadas · cada guardado en una transacción



Un intento de ingreso con un usuario o evento inexistente se informa en el momento pero no se guarda, porque no refiere a ninguna fila.

Datos para probar todos los casos

Si la base está vacía se cargan 8 usuarios, 5 artistas, 7 eventos y 16 accesos

Cuenta	Rol	Contraseña
`admin`	Administrador	`admin123`
`lferraro`, `lgabian`, `ncaggia` …	Usuario	`<cuenta>2024`
`tgomez`	Usuario inactivo	`tgomez2024`
`bandahorizonte`	Artista	`horizonte2024`
`djprisma`	Artista	`prisma2024`
`sunrust`	Artista	`sunrust2024`

Evento	Estado	Plan
100 Rock en Vivo	En curso (3 de 5)	Premium
101 Backstage Prisma Live	En espera	Artist Pass
102 Noche de Folklore	Pausado	Free
103 Luna en Vivo (Acústico)	Programado	Free
104 Festival de Verano 2025	Finalizado	Free
105 Fiesta Electrónica Cancelada	Cancelado	Premium
106 Sunrust - Transmisión permanente	Siempre en curso	Free

> El evento **106** termina en 2099: siempre se puede ingresar y ver la transmisión. Es el que usamos para la demo.

La API del servidor

Después de iniciar sesión, cada pedido lleva `Authorization: Bearer <token>`

Método y ruta	Acción
`POST / DELETE /api/sesion`	Inicia o cierra sesión
`GET /api/estado`	Todo lo que la cuenta puede ver
`POST /api/usuarios`	Registra un usuario
`POST /api/artistas`	Registra un artista
`POST /api/eventos`	Crea un evento
`POST /api/eventos/{id}/iniciar …`	Iniciar, pausar, reanudar, finalizar, cancelar
`POST /api/eventos/{id}/ingresos`	Solicita el ingreso
`POST /api/eventos/{id}/expulsiones`	Expulsa a un usuario
`GET /api/eventos/{id}/calendario`	Dirección del calendario
`GET /api/eventos/{id}/transmision`	Transmisión del evento

Rol	Recibe en `/api/estado`
Administrador	Todos los eventos, usuarios y registros
Artista	Solo sus eventos y sus conectados; ningún registro
Usuario	Todos los eventos, su cuenta y solo sus registros

Código	Excepción en el cliente
409 `acceso`	`AccesoDenegadoException`
401 / 403	`PermisoDenegadoException`
500	`PersistenciaException`
Sin conexión	`ConexionException`

El cliente lanza las mismas excepciones que el gestor local, así las vistas no distinguen entre los dos.

Roles y permisos

Igual que en la iteración anterior; se quitan guardar y cargar y se agregan calendario y transmisión

Acción	Usuario	Artista	Administrador
Solicitar ingreso a un evento	A su nombre	No	Sí
Ver eventos y su capacidad	Todos	Los propios	Todos
Agregar un evento a su calendario	Sí	Los propios	Sí
Ver la transmisión de un evento en curso	Si ingresó	Los propios	Sí
Crear un evento	No	A su nombre	Sí
Iniciar, pausar, reanudar, finalizar o cancelar	No	Los propios	Cualquiera
Ver usuarios conectados	No	Los propios	Sí
Registrar, activar o desactivar cuentas	No	No	Sí
Expulsar a un usuario	No	No	Sí
Ver registros de acceso	Los suyos	No	Todos

Calendarios y transmisiones

Calendarios

- Se selecciona un evento y se toca **Google Calendar** o **Apple Calendar**; la persona confirma allí.
- El evento queda al día: si cambia el horario o se cancela, se ve en el calendario.
- Cada evento se publica en `/calendario/<código>/<id>.ics`. El código es un HMAC de la cuenta: no se puede adivinar y no hace falta login.
- **Arranca solo con el programa:** en modo completo, un servidor aparte publica solo los `.ics` en la red local (puerto 8767); la API sigue cerrada a la red.
- Dirección: `uadebeats.url`, el túnel de Cloudflare (si `cloudflared` está instalado se abre solo) o la IP de la red.
- **Google** necesita https. **Apple** usa el túnel o `webcal://` en la red local (sin verificar en Mac).

Transmisiones

- Servicio aparte en Node.js que busca un recital del artista en YouTube y lo muestra a pantalla completa.
- Arranca en el minuto que corresponde según cuándo empezó el evento; no se puede pausar ni adelantar.
- Al autorizarse el ingreso a un evento en curso, se abre sola. También con «Ver transmisión».
- Se cierra si la persona cierra sesión, la expulsan o el evento termina. Arranca y se apaga junto con el programa.

Mejoras en la ventana

Sin clases nuevas: todo en `VentanaPrincipal` y los formularios separados en `Formularios`

Mejora	Detalle
En vivo	Los eventos en curso se marcan en rojo y negrita.
Doble clic	Acción según el rol: entrar al evento, abrir la transmisión o ver los conectados.
Confirmaciones	Antes de finalizar, cancelar, expulsar o desactivar.
Actualización automática	Las tablas se refrescan cada 5 segundos; en modo cliente se ven los cambios de otros.
Buscador	Filtra por título, estado o artista mientras se escribe.
Barra de estado	Mensajes en verde o rojo que se borran solos; menos botones y filas alternadas.

Cómo correrlo

Desde `TP/Grupo9_POO_EntregaFinal/Codigo` (en Windows el separador del classpath es `;`)

```
javac -encoding UTF-8 -cp "lib/*" -d out $(find src -name "*.java")

java --enable-native-access=ALL-UNNAMED -cp "out:lib/*" Main

java --enable-native-access=ALL-UNNAMED -cp "out:lib/*" Main servidor

java -cp "out:lib/*" Main cliente http://192.168.0.10:8765
```

Servidor público

`./servidor-publico.sh` abre un túnel de Cloudflare, toma la dirección https y arranca el servidor con ella. Muestra el comando para conectar un cliente.

Pruebas

`test/PruebasIteracion4.java`: 110 comprobaciones de Strategy, permisos, gestor local y remoto por HTTP y SQLite. Se corren con `java -ea`.

GRASP y SOLID en esta iteración

Concepto	Dónde se ve
Polimorfismo	Las dos Strategy: el gestor no pregunta el rol ni el proveedor de calendario
Variaciones Protegidas	`PoliticaVisibilidadEvento`, `VinculoCalendario`, `Persistencia`, `ProveedorTransmision`
Indirección	`GestorRemoto` y `ClienteTransmision` como intermediarios con la red
Fabricación Pura	Repositorios, `PoliticasVisibilidad`, `ControlTransmision`
SRP	La visibilidad sale del gestor; la transmisión pasa a `ControlTransmision`; los formularios salen de la ventana
OCP	Un calendario nuevo es una clase nueva, sin tocar el gestor
ISP	`ServicioEventos` dividido en sesión, consultas y comandos
DIP	Todo llega al gestor por constructor como interfaz; SQLite reemplazó a los TXT sin tocarlo

Limitaciones conocidas

Limitación	Detalle
Conexión sin cifrar	La API usa HTTP. Fuera de una red de confianza conviene un túnel con https o una VPN.
Base anterior	Una base creada antes de los hashes no es compatible: hay que borrarla para que se cree de nuevo.
Sesiones	Viven en memoria y no vencen por tiempo.
Concurrencia	La ventana del equipo servidor lee objetos sin bloqueo mientras un cliente remoto puede modificarlos.
Transmisiones	Sin clave de la API de YouTube, la búsqueda lee la página de resultados, que no es estable.
Fin del evento	`fechaHoraFin` es la ventana para ingresar, no la duración real. Viene del diseño original del modelo.

¿Preguntas?

UADE Beats · Grupo 10 · Proceso de Desarrollo de Software